BUSINESS ANALYTICS · COURSE PROJECT REPORT

skillet

An evidence-based skill recommender for software developers

Full report on the data, the mathematical modelling, the results and the interactive application.

DATA	Stack Overflow Developer Survey 2025 (2022–2024 for the trend analysis)
SAMPLE	19,210 working developers · 122 skills · 156 countries
LIVE APP	https://skillet.streamlit.app/
DATE	November 2026

Contents

1. Executive summary
2. Problem and objectives
3. Data
4. Exploratory analysis
5. Mathematical modelling - 5.1 Notation · 5.2 Why log pay · 5.3 Simple regression · 5.4 Multiple regression with country fixed effects · 5.5 Per-skill salary ROI · 5.6 Joint uplift of a learning plan · 5.7 Hypothesis tests · 5.8 Classification of job satisfaction · 5.9 PCA and K-means personas · 5.10 Factor analysis · 5.11 ANOVA across personas · 5.12 Skill co-occurrence network · 5.13 The recommender · 5.14 Skill map (t-SNE) · 5.15 Recommender evaluation · 5.16 Multi-year trend
6. Results
7. Worked example
8. The application
9. Model validity and the corrections made
10. Recommendations for decision makers
11. Assumptions and limitations
12. Reproducibility
13. Appendix: syllabus mapping and file map

1. Executive summary

Companies and individual developers spend a lot on training, but the choice of *what* to learn is mostly made on instinct or trend articles. **skillet** answers one question with statistical evidence:

> *Given my role, experience, country, AI-tool use and current stack, which five skills should I learn next, and what are they associated with in pay?*

The project combines three kinds of model:

Layer	Method	What it gives
Salary evidence	OLS regression of log USD pay on each skill, holding experience and country fixed	a % pay association, p-value and 95% CI for each of 121 skills
Personalisation	Profile-filtered, similarity-weighted K-nearest-neighbours on Jaccard distance	what developers *like you* know and want to learn
Plan valuation	One joint regression over the five recommended skills	the combined pay association of the whole plan, with a valid 95% CI

Supporting analyses (hypothesis tests, logistic regression, decision tree, Naive Bayes, PCA, K-means, hierarchical clustering, factor analysis, ANOVA, a co-occurrence network, t-SNE and a four-year trend) describe the population and feed the recommender.

Headline findings (19,210 working developers, 2025 survey):

- Experience alone explains 12.8% of the variance in log pay: about **+3.7% per year**.
- Experience, country and the 122 skills together explain **55.9%** (adjusted $R^2 = 0.553$).
- The highest-paying skills, holding experience and country fixed, are Scala (+35.1%), Snowflake (+33.0%), Terraform (+31.0%), ClickHouse (+30.7%) and Databricks SQL (+30.1%), all with $p < 10^{-12}$.
- **AI-tool users earn 9.9% more** than non-users at the same experience and country (95% CI +7.3% to +12.7%), even though the raw comparison points the other way. This is a Simpson's-paradox reversal caused by country.
- Job satisfaction is barely predictable from skills and work arrangement (best AUC = 0.569).
- Three developer personas exist: **.NET stack** (17.9%), **lean generalist stacks** (46.2%) and **modern JS / web tooling** (35.9%).
- On 500 held-out respondents the recommender recovers a hidden skill in its top five **78%** of the time. That beats a popularity baseline (72%) and a salary-only baseline (5%). Its picks are also the ones respondents most often say they want to learn.

2. Problem and objectives

2.1 Business problem

Learning and development (L&D) budgets are allocated with little evidence. There are hundreds of possible skills, and they differ in three ways:

1. their labour-market value (pay);
2. how relevant they are to a particular person's existing stack;
3. how much demand there is for them among comparable developers.

Generic "top skills of the year" lists ignore points 2 and 3. A salary ranking alone ignores 2: a front-end developer is told to learn Scala.

2.2 Objectives

- **Primary.** Recommend, for an individual profile, the top five next skills. Each recommendation comes with its estimated pay association, statistical significance, peer adoption and peer demand.
- **Secondary.**
- (a) Quantify the value of a whole learning plan with a correct confidence interval.
- (b) Identify developer personas for portfolio-level L&D planning.
- (c) Test common beliefs: do AI users earn more, and are they happier?
- (d) Track how skill premiums changed from 2022 to 2025.

2.3 Users

- Individual developers deciding what to learn.
- L&D and HR managers deciding what to fund.
- Course graders, who need to see the method and the evidence behind every number.

3. Data

3.1 Source

The Stack Overflow Developer Survey 2025 (`data/results.csv` , one row per respondent) is downloaded by `data/download.sh` . The 2022–2024 survey files are used only for the trend analysis (§5.16).

3.2 Cleaning (`src/preprocess.py`)

Step	Rule	Rationale
Population	`MainBranch` contains "developer by profession" **and** `Employment` is employed, independent, freelancer or self-employed	The question is about working developers
Experience	`WorkExp` , `YearsCode` : "Less than 1 year" → 0, "More than 50 years" → 51, then capped at 50	A few answers of 53–100 years are implausible and would have high leverage on the slope
Pay	`CompUSD` = `ConvertedCompYearly` (annual, USD at the survey exchange rate); $y = \log(1 + \text{CompUSD})$	`CompTotal` is in local currency (INR, JPY, ...) and cannot be compared across countries
Outliers	IQR rule on y: inlier if $Q_1 - 3\,\text{IQR} \le y \le Q_3 + 3\,\text{IQR}$	Removes impossible pay entries (e.g. $1 per year) without trimming genuine high earners
Satisfaction	`JobSat` (0–10) → `JobSat_num` ; `JobSat_high` = 1 if ≥ 8	Gives a nearly balanced binary target (50.3% positive)
AI use	`ai_user` = 1 if `AISelect` starts with "Yes"	
Skills	Multi-select answers exploded into 0/1 columns: languages (top 60), databases (25), platforms (20), web frameworks (15), dev environments (10), AI models (10)	122 skill columns in total
Roles	`DevType` exploded into 32 role dummies	Used by the peer filter
Missing	Rows missing `WorkExp` , `CompTotal` or `JobSat` are dropped	The essentials for the models

The 253 respondents who reported pay that the survey could not convert to USD are kept with y = NaN. The skill-only analyses (clustering, factor analysis, network, satisfaction) use them; every salary model filters them out. This keeps the personas identical whether or not pay is known.

3.3 Resulting samples

Sample	Rows	Used by
Processed dataset	**19,210** developers, 170 columns, 156 countries	clustering, factor analysis, network, satisfaction models, recommender peer pool
With USD pay	18,957	raw t-test, ANOVA
Salary model sample (USD pay, IQR inlier)	**18,388** developers, 151 countries	all regressions, peer pay premium

3.4 Variables

Symbol	Column	Type
y_i	`log_comp` = log(1 + annual USD pay)	continuous
x_i	`WorkExp` , years of professional experience (0–50)	continuous
$c(i)$	`Country`	categorical (151 or 156 levels)
s_{ij}	skill j known by developer i	binary, $j = 1, \ldots, 122$
a_i	`ai_user`	binary
h_i	`JobSat_high`	binary
W_i	the set of skills i *wants* to work with (`*WantToWorkWith`)	set

4. Exploratory analysis

The figures are generated in `assets/` by `notebooks/01_eda.py` and the analysis modules.

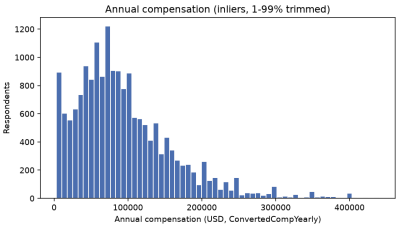
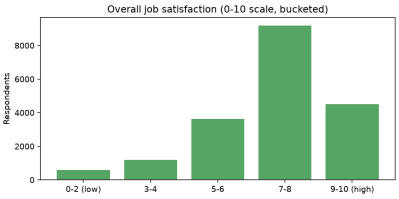
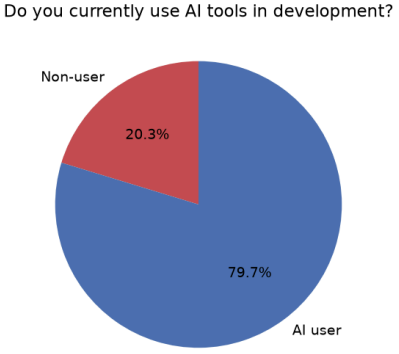
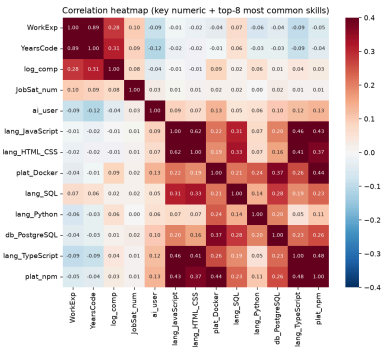
Figure	What it shows
	Distribution of pay: USD pay is strongly right-skewed, which motivates modelling log pay
	Log pay rises with experience, with large spread at every level
	Log pay by experience band
	Job satisfaction peaks at 7–8 of 10 (48% of answers); 50.3% answer 8 or more
	About 80% of working developers use AI tools

Figure	What it shows
	Correlations between the numeric variables

Two observations from exploration shaped the models:

1. **Country dominates pay.** USD pay differs by an order of magnitude between countries, so any model that does not control for country attributes country differences to skills.
2. **Skills co-occur.** Docker, Kubernetes and AWS, for example, are usually known together, so their separate effects overlap (see §5.6).

5. Mathematical modelling

5.1 Notation

- Developers are indexed $i = 1, \dots, n$ and skills $j = 1, \dots, p$.
- $\mathbf{s}_i \in \{0,1\}^p$ is developer i's skill vector.
- $S_i = \{j : s_{ij} = 1\}$ is the same information as a set.
- $\mathbf{1}[\cdot]$ is the indicator function.
- $D_{ic} = \mathbf{1}[c(i) = c]$ is the dummy for country c. The United States, the largest country, is the reference level and has no dummy.

5.2 Why model log pay

The response is $y_i = \log(1 + \text{CompUSD}_i)$, for three reasons.

1. **Skew.** Pay is right-skewed and its variance grows with its level, while log pay is roughly symmetric. That suits OLS, whose inference assumes errors of constant variance.
2. **Interpretation as percentages.** With $y = \log(\text{pay})$, a coefficient β on a binary variable multiplies pay by e^{β}. So the percentage effect is

$$\%\Delta = (e^{\beta} - 1) \times 100.$$

A 95% interval transforms the same way, because the map is monotone:

$$\left[(e^{\hat\beta - t_{0.975}\,\mathrm{SE}} - 1)\cdot 100,\;\; (e^{\hat\beta + t_{0.975}\,\mathrm{SE}} - 1)\cdot 100\right].$$

All percentages in this report and the app use this transformation, not 100β. The two agree only for small β. 3. **Additivity across countries.** Country effects become multiplicative pay levels. A skill premium is then a percentage, which is comparable between a developer in India and one in the US.

5.3 Simple linear regression (SLR)

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \qquad \varepsilon_i \sim (0, \sigma^2).$$

The OLS estimates minimise $\sum_i (y_i - \beta_0 - \beta_1 x_i)^2$:

$$\hat\beta_1 = \frac{\sum_i (x_i - \bar x)(y_i - \bar y)}{\sum_i (x_i - \bar x)^2}, \qquad \hat\beta_0 = \bar y - \hat\beta_1 \bar x.$$

Result (n = 18,388):

- $\hat\beta_0$ = 10.661 and $\hat\beta_1$ = 0.0362 log points per year (p ≈ 0).
- That is $(e^{0.0362} - 1)$ = **+3.7% pay per year of experience**.
- R^2 = 0.128 and F = 2,705.

Goodness of fit is measured by

$$R^2 = 1 - \frac{\sum_i (y_i - \hat y_i)^2}{\sum_i (y_i - \bar y)^2}.$$

5.4 Multiple regression with country fixed effects

The full descriptive model puts every skill and every country in one equation:

$$y_i = \beta_0 + \beta_x x_i + \sum_{j=1}^{p} \gamma_j s_{ij} + \sum_{c \neq \text{US}} \alpha_c D_{ic} + \varepsilon_i.$$

In matrix form, $\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}$, the OLS estimate and its covariance are

$$\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}, \qquad \widehat{\text{Var}}(\hat{\boldsymbol{\beta}}) = \hat{\sigma}^2 (X^\top X)^{-1}, \qquad \hat{\sigma}^2 = \frac{\|\mathbf{y} - X\hat{\boldsymbol{\beta}}\|^2}{n - k}.$$

Country fixed effects. Each α_c absorbs the average log-pay level of country c. Every other coefficient is therefore estimated from *within-country* variation: it compares developers in the same country. Without this control a skill that is common in high-pay countries (e.g. Snowflake in the US) would look valuable only because of where its users live.

Result:

- R^2 = **0.559**, adjusted R^2 = 0.553, k = 274 parameters (constant, experience, 122 skills and 150 country dummies), F-test p ≈ 0.
- adj-$R^2 = 1 - (1 - R^2)\frac{n-1}{n-k}$ penalises the 274 parameters.

Multicollinearity check. Variance inflation factors are computed as

$$\text{VIF}_j = \frac{1}{1 - R_j^2},$$

where R_j^2 comes from regressing variable j on the others. On the sampled columns they lie between 1.0 and 1.6, well below the usual concern threshold of 5–10. So the individual estimates are stable. Correlated skills still share credit, though, which §5.6 handles.

5.5 Per-skill salary ROI (the recommender's salary prior)

For each skill j, one regression is fitted separately:

$$y_i = \beta_0 + \beta_x x_i + \beta_j s_{ij} + \sum_{c} \alpha_c D_{ic} + \varepsilon_i.$$

$\hat{\beta}_j$ is the **salary ROI** of skill j: the expected log-pay difference between developers who know the skill and those who don't, at the same experience in the same country.

The quantities reported for each skill (`data/skill_roi.csv`) are:

- **t statistic.** $t_j = \hat{\beta}_j / \text{SE}(\hat{\beta}_j)$, with $n - k$ degrees of freedom.
- **p-value.** $2\,P(T_{n-k} > |t_j|)$ for $H_0 : \beta_j = 0$.
- **95% CI.** $\hat{\beta}_j \pm t_{0.975,\,n-k}\,\text{SE}(\hat{\beta}_j)$, then transformed to percent (§5.2).
- **Inclusion rule.** A skill is estimated only if at least 50 developers in the model sample know it. 121 of the 122 skills qualify.

Why one regression per skill? Each per-skill $\hat{\beta}_j$ answers the question a learner actually asks: what is skill j associated with, compared with developers like me who lack it? The kitchen-sink model of §5.4 answers a different question, the effect of j with every other skill held fixed. That makes

single coefficients hard to read when skills come in bundles. The per-skill estimates are the right *ranking* signal. They are the wrong thing to *add up*, which is why plans are valued with the joint model of §5.6.

5.6 Joint uplift of a learning plan

For a plan $P = \{j_1, \dots, j_m\}$ (normally the five recommended skills), one regression with all of them together is fitted:

$$y_i = \beta_0 + \beta_x x_i + \sum_{j \in P} \beta_j s_{ij} + \sum_c \alpha_c D_{ic} + \varepsilon_i.$$

The combined effect is the linear contrast

$$\theta = \mathbf{1}_P^\top \boldsymbol{\beta} = \sum_{j \in P} \beta_j, \qquad \widehat{\mathrm{Var}}(\hat\theta) = \mathbf{1}_P^\top \, \widehat{\Sigma}_{PP} \, \mathbf{1}_P = \sum_{j \in P} \sum_{k \in P} \widehat{\mathrm{Cov}}(\hat\beta_j, \hat\beta_k),$$

where $\mathbf{1}_P$ is 1 on the plan's coefficients and 0 elsewhere. It is tested with $t = \hat\theta / \mathrm{SE}(\hat\theta)$ (`statsmodels t_test`), and the reported values are

$$\text{total uplift} = (e^{\hat\theta} - 1) \cdot 100, \qquad 95\%\ \text{CI} = \left(e^{\hat\theta \pm t_{0.975}\mathrm{SE}(\hat\theta)} - 1\right) \cdot 100.$$

Why this matters. Adding up separate per-skill coefficients double-counts. A developer who knows Kubernetes almost always knows Docker, so each of the two stand-alone coefficients already contains much of the other's premium.

- **Docker + Kubernetes + AWS + Python + Terraform.** The naive sum of stand-alone effects is +152%. The joint model gives **+57%** (95% CI +51% to +62%).
- **Adding up interval bounds gives no interval.** Summing per-skill interval bounds does not produce a confidence interval; the covariance terms above are required.

5.7 Hypothesis tests (α = 0.05)

(a) Welch's two-sample t-test, AI users (u) versus non-users (v):

$$t = \frac{\bar y_u - \bar y_v}{\sqrt{s_u^2/n_u + s_v^2/n_v}}, \qquad \nu \approx \frac{(s_u^2/n_u + s_v^2/n_v)^2}{\frac{(s_u^2/n_u)^2}{n_u - 1} + \frac{(s_v^2/n_v)^2}{n_v - 1}}.$$

Welch's version does not assume equal variances.

(b) Controlled AI pay gap. The same difference is estimated holding experience and country fixed:

$$y_i = \beta_0 + \delta \, a_i + \beta_x x_i + \sum_c \alpha_c D_{ic} + \varepsilon_i, \qquad H_0 : \delta = 0.$$

(c) Chi-square test of independence for two categorical variables, with an $r \times k$ contingency table O and expected counts $E_{ab} = (\text{row}_a \cdot \text{col}_b)/N$:

$$\chi^2 = \sum_{a,b} \frac{(O_{ab} - E_{ab})^2}{E_{ab}}, \quad \mathrm{df} = (r-1)(k-1), \qquad V = \sqrt{\frac{\chi^2}{N\,(\min(r,k) - 1)}} \ \text{(Cramér's V)}.$$

Results:

Test	Statistic	p	Decision
Log pay, AI users vs non-users (Welch)	t = −6.15	8.3×10^{-10}	Reject H_0: raw gap favours **non-users**
Same, controlled (OLS, n = 18,388)	t = 7.53, δ = 0.095 → **+9.9%** [+7.3%, +12.7%]	5.2×10^{-14}	Reject H_0: AI users earn **more**
Job satisfaction, AI users vs non-users (Welch)	t = 3.36 (7.21 vs 7.08)	7.8×10^{-4}	Reject H_0: AI users slightly more satisfied
AI use × remote work (χ^2, df = 5)	χ^2 = 10.59, V = 0.023	0.060	Fail to reject H_0
Primary role × knows JavaScript (χ^2, df = 13)	χ^2 = 2,416, V = 0.355	$< 10^{-30}$	Reject H_0: strong association

Simpson's paradox. The raw t-test and the controlled regression disagree in sign. AI-tool use is more common among developers in lower-paying countries. Pooled together, AI users therefore look lower-paid. Within any one country, at equal experience, they earn about 10% more. The controlled estimate answers the relevant question; the raw one is confounded by country.

5.8 Classification of job satisfaction

Target: h_i = 1 if job satisfaction is at least 8 of 10 (50.3% positive).

Features (41 in all):

- experience and AI use;
- five remote-work dummies, mapped to the exact 2025 answers;
- four organisation-size dummies;
- the 30 most common skills.

Validation: a stratified 75/25 train/test split (`random_state = 42`).

Logistic regression. Features are standardised, $z = (x - \mu)/\sigma$, and the model is

$$P(h_i = 1 \mid \mathbf{z}_i) = \sigma(\mathbf{w}^\top \mathbf{z}_i + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{z}_i + b)}}.$$

It is fitted by minimising the class-weighted log-loss with an L2 penalty (C = 1):

$$\mathcal{L} = -\sum_i \omega_{h_i} \left[h_i \log \hat{p}_i + (1 - h_i) \log(1 - \hat{p}_i) \right] + \tfrac{1}{2C} \|\mathbf{w}\|^2, \qquad \omega_k = \frac{n}{2\, n_k}.$$

Here n_k is the number of training examples in class k.

Decision tree (`max_depth` = 6, balanced class weights). Each split is chosen to minimise the weighted Gini impurity $G = 1 - \sum_k p_k^2$ of the child nodes. Feature importance is the total impurity reduction a feature contributes.

Gaussian Naive Bayes. Assumes the features are conditionally independent given the class, with Gaussian likelihoods:

$$P(h \mid \mathbf{x}) \propto P(h) \prod_f \mathcal{N}(x_f;\, \mu_{fh}, \sigma^2_{fh}).$$

Evaluation.

- The ROC curve plots the true-positive rate against the false-positive rate at every threshold. AUC is the probability that a random positive is scored above a random negative.
- The operating threshold maximises Youden's $J = \text{TPR} - \text{FPR}$.

Model	AUC	F1	Accuracy
Logistic regression	0.565	0.614	0.548
Naive Bayes	**0.569**	0.580	0.551
Decision tree	0.560	0.461	0.553

What the models find. All three models agree that satisfaction is only weakly predictable from skills and work arrangement.

- The strongest logistic coefficients are experience (+0.21), working at a small organisation (+0.21) and in-person or hybrid-in-person work (−0.17 to −0.19).
- The tree ranks experience as by far the most important feature (0.44).
- This is consistent with research showing that satisfaction depends mostly on factors the survey does not measure (manager, team, autonomy). It is reported as a finding.
- The per-skill logistic coefficients (`data/satisfaction_coefs.csv`) are shown in the app as a secondary signal.

5.9 Personas: PCA, K-means and hierarchical clustering

Input. The 86 skills known by at least 5% of developers, standardised column-wise. Rare skills are dropped because they add noise dimensions that dominate distance-based clustering.

PCA. PCA finds orthogonal directions of maximal variance: the eigenvectors of the covariance matrix $\Sigma = \frac{1}{n} Z^\top Z$. Projected onto the first q eigenvectors V_q, the scores are $T = ZV_q$. The smallest q with

$$\frac{\sum_{l \le q} \lambda_l}{\sum_l \lambda_l} \ge 0.80$$

is kept, which gives q = **51** components (80.3% of variance).

K-means on the PCA scores minimises the within-cluster sum of squares

$$\min_{C_1,\dots,C_k} \sum_{r=1}^{k} \sum_{i \in C_r} \lVert \mathbf{t}_i - \boldsymbol{\mu}_r \rVert^2$$

using Lloyd's algorithm with 10 random restarts.

Choosing k. k is chosen over $3 \le k \le 8$ by the mean silhouette

$$\bar{s} = \frac{1}{n} \sum_i \frac{b_i - a_i}{\max(a_i, b_i)}.$$

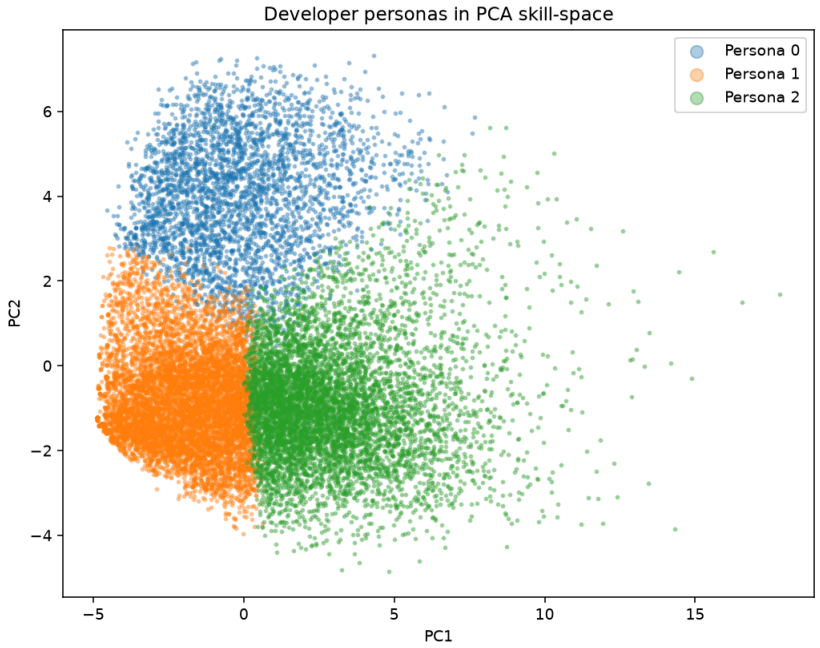
a_i is the mean distance from i to its own cluster and b_i the mean distance to the nearest other cluster. The best is **k = 3** ($\bar{s}$ = 0.085). Low silhouettes are normal for sparse high-dimensional binary data; the clusters are still clearly interpretable.

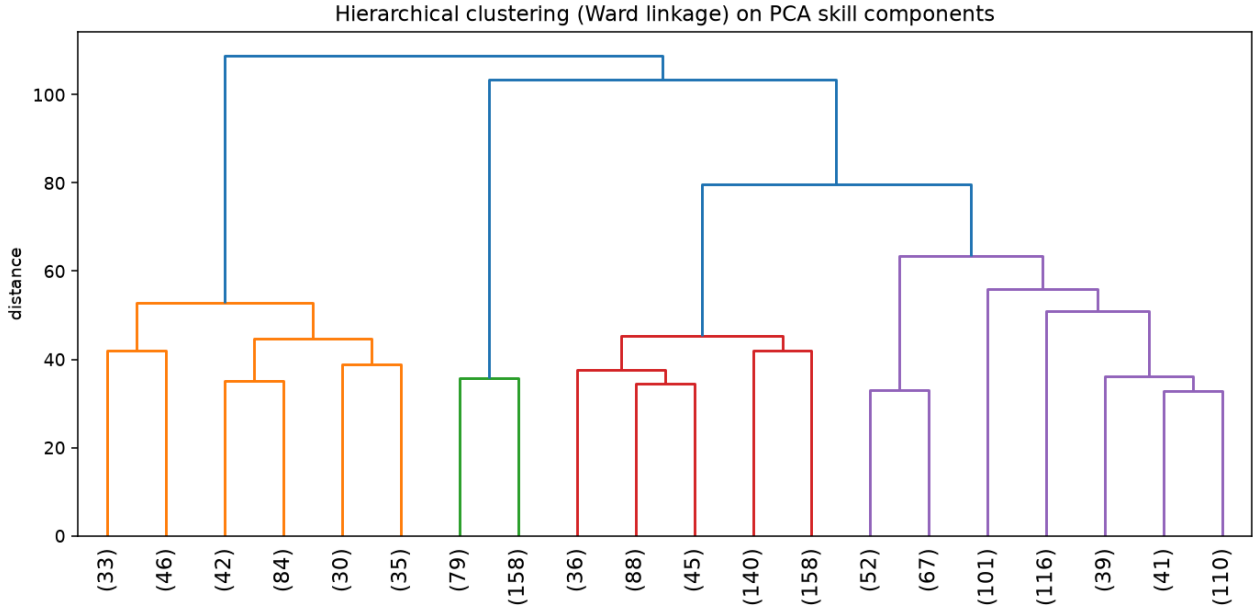
Hierarchical clustering. Ward linkage on a 1,500-row subsample is used only for the dendrogram. At each step Ward merges the pair of clusters that increases total within-cluster variance least.

Persona profiles. Each cluster is described by its distinguishing skills, ranked by the lift

$$\text{lift}_{rj} = \frac{\text{share of cluster } r \text{ knowing } j}{\text{share of all developers knowing } j}.$$

Persona	Share	Distinguishing skills (lift)
.NET stack	17.9%	ASP.NET (4.8×), ASP.NET Core, NuGet, C#, Visual Studio, SQL Server
Lean generalist stacks	46.2%	Small stacks (≈10 skills vs 21–27 in the other two); C++, C, Swift, Java, Oracle at only ≈1.1× lift
Modern JS / web tooling	35.9%	pnpm, Express, Yarn, Webpack, Vite





5.10 Exploratory factor analysis

PCA summarises variance. Factor analysis instead models *latent causes* of co-occurrence:

$$\mathbf{z} = \Lambda \mathbf{f} + \boldsymbol{\epsilon}, \qquad \operatorname{Cov}(\mathbf{z}) = \Lambda\Lambda^{\top} + \Psi,$$

where Λ ($86 \times m$) holds the loadings, $\mathbf{f}$ the m common factors and Ψ the diagonal unique variances.

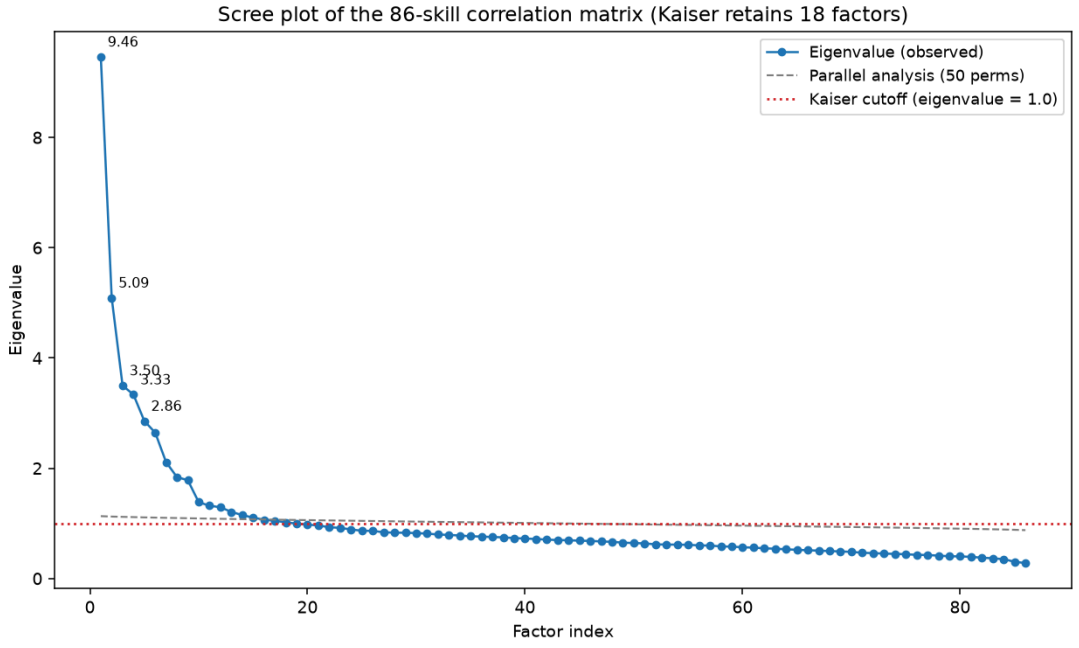
Number of factors. The Kaiser criterion (correlation-matrix eigenvalues above 1) gives m = **18**. A parallel-analysis baseline (eigenvalues of random data of the same size) suggests 15. Eighteen are kept, and they explain 50.1% of the variance.

Rotation. Varimax rotation maximises the variance of the squared loadings within each factor, so that each skill loads strongly on few factors:

$$\max_{R} \sum_{q} \left[\tfrac{1}{p} \sum_{j} \lambda_{jq}^{4} - \left(\tfrac{1}{p} \sum_{j} \lambda_{jq}^{2} \right)^{2} \right].$$

The factors read as technology ecosystems:

- F1: JavaScript front-end (React, Node.js, Next.js)
- F2: .NET (ASP.NET Core, C#, NuGet)
- F3: Java enterprise (Java, Spring Boot, Maven)
- F4: OpenAI tooling
- F5: Linux CLI and DevOps (APT, Docker, Pip)
- F7: native/systems (C++, C, Make)
- F8: mobile (Android Studio, Dart, Swift)



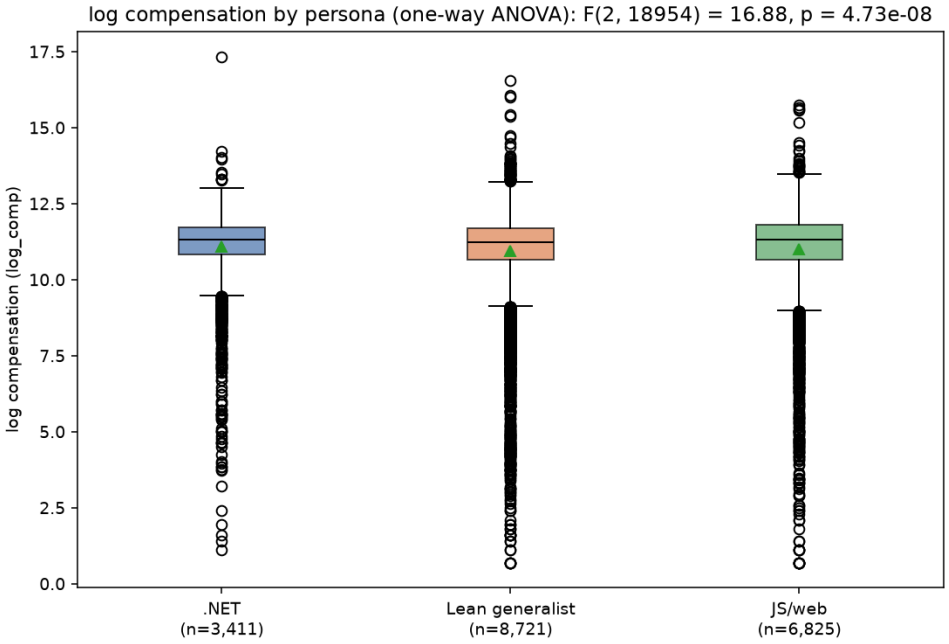
5.11 One-way ANOVA of pay across personas

$$F = \frac{\mathrm{SS}_B/(k-1)}{\mathrm{SS}_W/(N-k)}, \quad \mathrm{SS}_B = \sum_r n_r(\bar{y}_r - \bar{y})^2, \quad \mathrm{SS}_W = \sum_r \sum_{i \in r} (y_i - \bar{y}_r)^2, \quad \eta^2 = \frac{\mathrm{SS}_B}{\mathrm{SS}_B + \mathrm{SS}_W}.$$

Result:

- F(2, 18,954) = 16.88, p = 4.7×10^{-8}, $\eta^2 = 0.0018$.
- Mean log pay: .NET 11.10, JS/web 11.00, lean generalist 10.94.
- Pairwise Welch t-tests with a Bonferroni correction ($p_{\text{adj}} = \min(1, 3p)$) are all significant. The largest gap is .NET vs lean generalist ($p_{\text{adj}} = 4.0\times10^{-10}$).

This test is raw: it does not control for country, and η^2 shows that the persona explains only 0.2% of pay variance. The persona describes the *kind* of stack, not its value; value comes from the per-skill regressions.

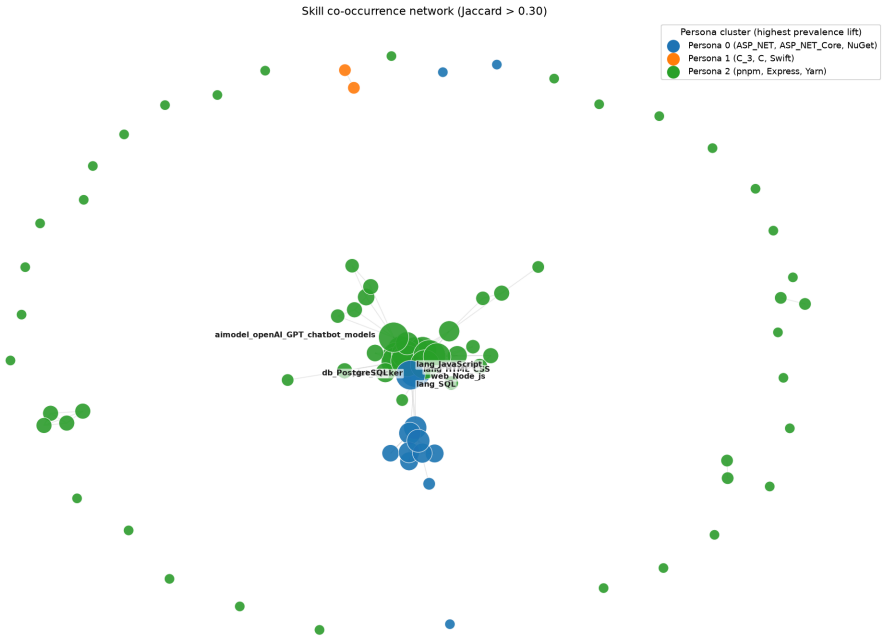


5.12 Skill co-occurrence network

Skills are nodes. An edge joins two skills when the Jaccard similarity of their user sets exceeds 0.30:

$$J(j,k) = \frac{|U_j \cap U_k|}{|U_j \cup U_k|}, \qquad U_j = \{i : s_{ij} = 1\}.$$

- **Edges.** The 86-node graph has 182 edges and 38 connected components.
- **Hubs.** Degree centrality, $\deg(v)/(N-1)$, ranks the most-connected skills: npm (0.235), Docker (0.224), SQL (0.212), PostgreSQL (0.200) and OpenAI chat models (0.200).
- **Modularity.** Modularity is $Q = \frac{1}{2M}\sum_{uv}\left[A_{uv} - \frac{k_u k_v}{2M}\right]\delta(c_u, c_v)$.
- Measured against the three personas, Q = 0.21.
- Greedy modularity maximisation finds communities with Q = 0.38.
- The skill graph is therefore organised by technical ecosystem more sharply than the three personas capture.



5.13 The recommender (`src/recommend.py`)

The input is a profile (role ρ, experience $x^\star$, country $c^\star$, AI use $a^\star$) and a current stack $S^\star$, with binary vector $\mathbf{s}^\star$.

Step 1: profile filter. The peer pool starts with every requested filter:

$$\mathcal{P} = \{\, i : \text{role}_i \ni \rho,\;\; c(i) = c^\star,\;\; |x_i - x^\star| \le 10,\;\; a_i = a^\star \,\}.$$

While $|\mathcal{P}| < 50$, filters are dropped one at a time in the fixed order AI use → experience → country → role, and the dropped filters are reported in the app. The order relaxes the least informative condition first.

Step 2: nearest neighbours by Jaccard distance. For each $i \in \mathcal{P}$:

$$d_J(\mathbf{s}^\star, \mathbf{s}_i) = 1 - \frac{|S^\star \cap S_i|}{|S^\star \cup S_i|}.$$

Jaccard ignores shared *absences*. With 122 skills, two developers who both lack 110 of them are not thereby similar, and Euclidean or Hamming distance would say they are. The k = 300 closest members of $\mathcal{P}$ form the neighbour set N.

Step 3: similarity weights. Each neighbour is weighted by similarity:

$$w_i = \frac{1 - d_J(\mathbf{s}^\star, \mathbf{s}_i)}{\sum_{l \in N} \big(1 - d_J(\mathbf{s}^\star, \mathbf{s}_l)\big)}, \qquad \sum_{i \in N} w_i = 1.$$

Many filtered pools have fewer than 300 members. With equal weights the whole pool would then count equally whatever the user's stack. Weighting keeps the stack in charge.

Step 4: peer signals. For each candidate skill $j \notin S^\star$:

$$\text{prev}(j) = \sum_{i \in N} w_i \, s_{ij}, \qquad \text{want}(j) = \sum_{i \in N} w_i \, \mathbf{1}[\, j \in W_i \,].$$

- $\text{prev}(j)$ is the similarity-weighted share of peers who **know** j.

- $\text{want}(j)$ is the share who say they **want to work with** it. This is a demand signal from the survey's separate `*WantToWorkWith` questions.

Step 5: score.

$$\text{score}(j) = \text{prev}(j) + 0.5\,\hat{\beta}_j + 0.3\,\text{want}(j) + 0.05\,\mathbf{1}[p_j < 0.05] + 0.02\,\mathbf{1}[a^\star = 1 \wedge j \text{ is an AI model}],$$

where $\hat{\beta}_j$ is the per-skill salary ROI from §5.5 in log points.

The terms are on comparable scales: shares are in [0, 1], and $\hat{\beta}$ ranges roughly from −0.3 to +0.3. In words, the score is relevance (do people like me use it?), plus value (is it associated with higher pay at my experience and country?), plus demand (do people like me want it?), plus a small bonus for statistically reliable value.

Constraints. A skill is eligible only if

$$j \notin S^\star, \qquad \text{prev}(j) > 0.02, \qquad \hat{\beta}_j > 0.$$

Negative-ROI skills (e.g. PHP, −19.8%) still appear in the ROI chart but are never recommended as upskilling. The top five by score are returned with their evidence.

Peer pay context. For any group G the app also reports:

- the median USD pay of G;
- the **premium** of G: the (weighted) mean residual from the experience + country model,

$$\text{premium}(G) = \sum_{i \in G} w_i \left(y_i - \hat{y}_i^{(x,c)}\right), \qquad \hat{y}^{(x,c)} = \hat{\beta}_0 + \hat{\beta}_x x_i + \hat{\alpha}_{c(i)}.$$

A positive premium means the user's peers earn more than developers with the same experience in the same country. This number is shown but deliberately **not** part of the score: it is the same for every candidate skill, so it cannot change the ranking.

Cold start. With an empty stack there is no neighbourhood. Skills known by at least 3% of the filtered pool are then ranked by

$$\hat{\beta}_j + 0.3\,\overline{s_{\cdot j}}^{\mathcal{P}} + 0.05\,\mathbf{1}[p_j < 0.05] + 0.02\,\mathbf{1}[\text{AI}].$$

Across the whole survey this gives Docker, Kubernetes, Terraform, AWS and Snowflake.

Persona assignment. The user's stack, restricted to the 86 cluster skills, is passed through the saved scaler → PCA → K-means model. The persona shown is exactly the cluster the analysis of §5.9 would assign. A stack with none of those skills gets the largest persona, and the app marks it as "add your stack".

5.14 Skill map (t-SNE, `src/skymap.py`)

The sky chart in the app places all 122 skills in 2-D so that skills used together sit together.

- **Distances.** $D_{jk} = 1 - J(j,k)$ (§5.12) for every pair of skills.
- **Embedding.** t-SNE turns distances into neighbour probabilities $p_{k|j} \propto \exp(-D_{jk}^2 / 2\sigma_j^2)$, with σ_j set by a perplexity of 12. It then finds 2-D points $\mathbf{u}_j$ whose Student-t similarities $q_{jk} \propto (1 + \|\mathbf{u}_j - \mathbf{u}_k\|^2)^{-1}$ minimise $\text{KL}(P\|Q) = \sum_{j \neq k} p_{jk} \log \frac{p_{jk}}{q_{jk}}$.
- **Scaling.** The seed is fixed (42), and coordinates are centred and scaled to [−1, 1].

- **Reading the map.** Only *neighbourhoods* are meaningful in t-SNE. Global distances and axis directions are not, so the chart has no axis labels.

5.15 Recommender evaluation (`src/evaluate.py`)

The recommender is tested offline on real respondents it has not seen in their own peer pool.

Sample. 1,000 respondents are drawn at random from those with at least six skills, at least two recommendable skills (positive ROI) and a non-empty want list. They are split 500 validation / 500 test. Each test respondent is removed from the peer pool and recommends from their own role, country, experience and AI use, exactly as the app would.

Task A, hidden-skill recovery. Two of the respondent's recommendable skills H ($|H| = 2$) are hidden, and five skills R are recommended from the rest:

$$\text{hit@5} = \mathbf{1}[\,R \cap H \neq \varnothing\,], \qquad \text{recall@5} = |R \cap H|/|H|.$$

Task B, wanted skills. From the full stack:

$$\text{want-precision@5} = |R \cap W_i|/5.$$

Value. The mean stand-alone salary effect of R.

Uncertainty.

- Each metric's 95% interval is a percentile bootstrap over respondents (2,000 resamples).
- A comparison of two methods bootstraps the paired per-respondent difference.
- The weights were searched on a grid (ROI weight ∈ {0, 0.25, 0.5, 1, 2}, demand weight ∈ {0, 0.15, 0.3, 0.6, 1, 1.5, 2}) on the validation half only.

Test-set results:

Method	hit@5	recall@5	want-precision@5	Mean salary effect
Popularity (most common skills)	0.718 [0.678, 0.756]	0.448	0.126 [0.111, 0.141]	+9.5%
Salary only (highest ROI)	0.054 [0.034, 0.074]	0.027	0.052 [0.042, 0.062]	+31.6%
KNN, prevalence only	0.786 [0.750, 0.822]	0.521	0.137 [0.123, 0.151]	+10.2%
KNN, chosen weights (0.5, 0.3)	**0.780** [0.744, 0.818]	0.508	**0.158** [0.144, 0.174]	**+12.1%**
KNN, heaviest grid point (2.0, 2.0)	0.746 [0.708, 0.782]	0.465	0.188 [0.172, 0.206]	+14.4%

Interpretation:

- The KNN beats both baselines.
- Adding salary and demand to plain prevalence raises want-precision (0.137 → 0.158) and plan value (+10.2% → +12.1%) at no meaningful cost in hit rate.
- The heaviest weights are a genuine **trade-off**. Paired against the chosen weights they gain +3.0 points of want-precision [+2.2, +3.9] and +2.4 points of salary value, but lose 3.4 points of hit rate [−5.6, −1.2].
- The chosen weights sit on the accuracy/value frontier. Heavier weights would be a product choice (more aspirational picks), not a correction.

5.16 Multi-year trend, 2022 → 2025 (`src/trend.py`)

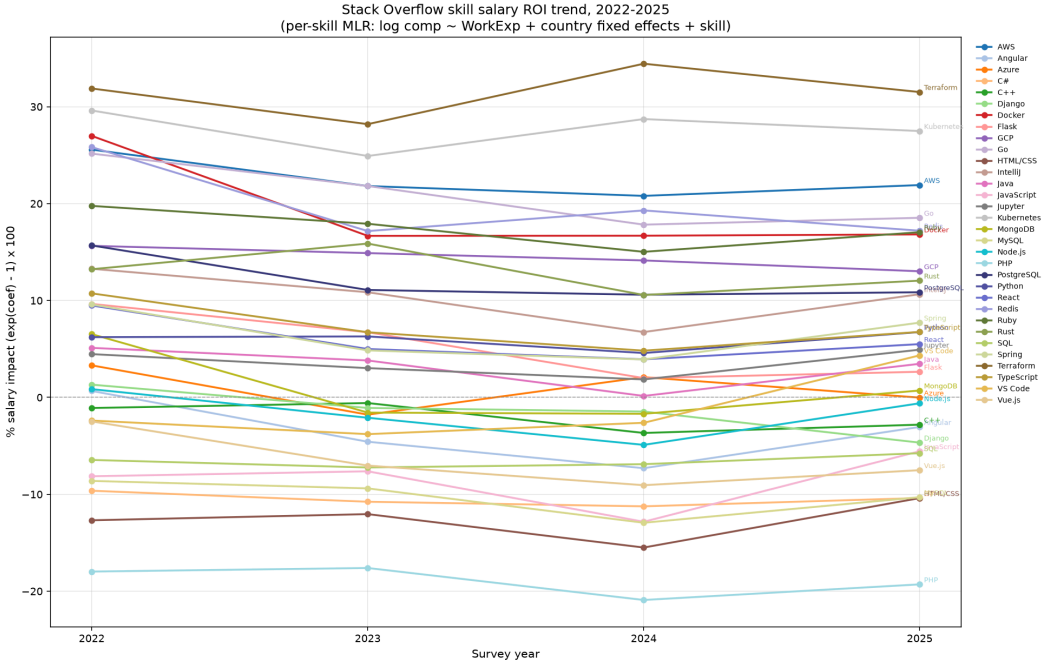
The per-skill model of §5.5 is re-fitted on each survey year. It uses the 32 skills present in all four years, USD pay, country fixed effects and the same population filter. Year samples: 25,249 / 30,739 / 15,163 / 20,472.

Harmonising names. Skill names change across survey years (e.g. "React.js" in 2022 became "React"; Docker moved from the tools question to the platforms question). A token map aligns them, recorded in `data/trend_summary.json` .

Findings:

- **Stable leaders.** Terraform (+31.9% → +31.5%) and Kubernetes lead every year.
- **Fading specialist premiums.** Docker fell from +27.0% to +16.8% (−10.2 points), Redis from +25.8% to +17.2% and Go from +25.2% to +18.5%, consistent with these skills becoming mainstream.
- **Few risers.** VS Code is the only gain above three points (+6.7).

These are within-year estimates on different respondents each year. Changes mix real market movement with changes in who answers the survey.

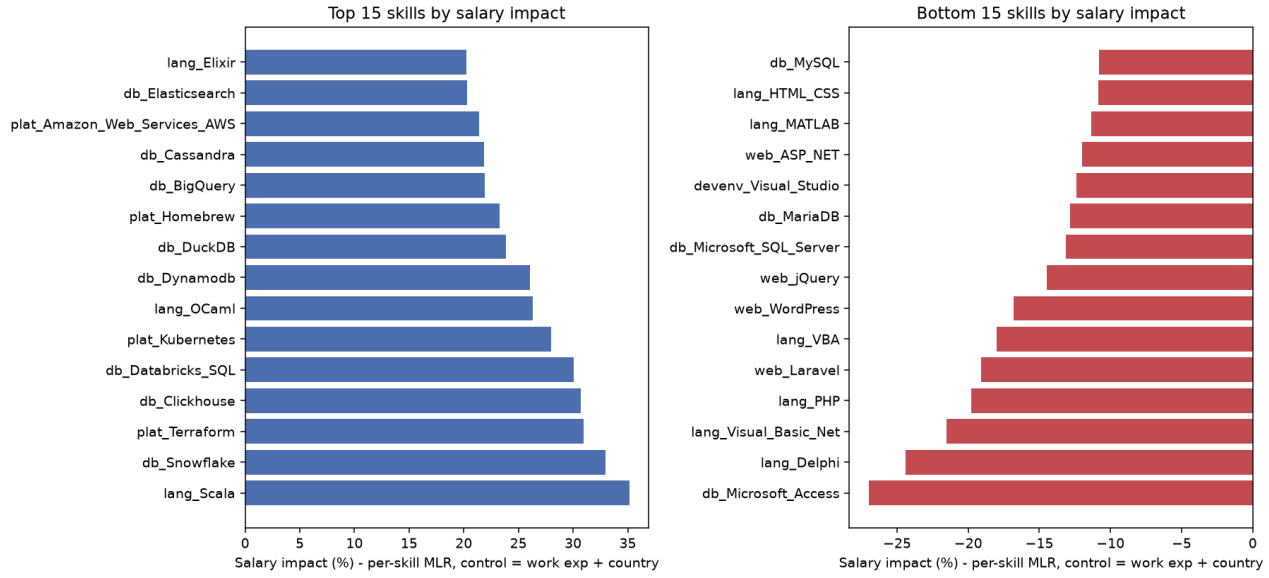


6. Results

6.1 Highest and lowest salary ROI (per-skill model, §5.5)

Rank	Skill	Pay association	95% CI	p	Users
1	Scala	+35.1%	+27.0% to +43.8%	2.6×10^{-21}	471
2	Snowflake	+33.0%	+26.1% to +40.2%	4.2×10^{-26}	665
3	Terraform	+31.0%	+27.5% to +34.5%	9.4×10^{-87}	2,958
4	ClickHouse	+30.7%	+21.8% to +40.2%	1.2×10^{-13}	366
5	Databricks SQL	+30.1%	+22.4% to +38.2%	2.8×10^{-17}	493
…					
−5	Laravel	−19.1%	−22.3% to −15.8%	5.8×10^{-25}	1,203
−4	PHP	−19.8%	−21.9% to −17.7%	5.7×10^{-61}	3,144
−3	VB.NET	−21.5%	−25.5% to −17.4%	5.0×10^{-20}	694
−2	Delphi	−24.4%	−29.7% to −18.7%	6.1×10^{-14}	351
−1	Microsoft Access	−27.0%	−31.5% to −22.2%	5.0×10^{-22}	452

Full table: `data/skill_roi.csv` (121 skills, 98 significant at 5%).



6.2 Summary of all models

Model	Key result
SLR, log pay ~ experience	+3.7% per year, $R^2 = 0.128$
MLR, + country FE + 122 skills	$R^2 = 0.559$, adj-$R^2 = 0.553$
AI pay gap, controlled	+9.9% [+7.3%, +12.7%] (raw comparison reverses)
Satisfaction classifiers	AUC 0.560–0.569: weakly predictable

Model	Key result
K-means personas	k = 3, silhouette 0.085, 51 PCA components
Factor analysis	18 factors, 50.1% of variance, one per ecosystem
ANOVA, pay by persona	$F = 16.9$, $p < 10^{-7}$, $\eta^2 = 0.002$
Network	182 edges, hubs npm / Docker / SQL; modularity 0.38
Recommender	hit@5 = 0.78, want-precision@5 = 0.158; beats baselines

7. Worked example

A full-stack developer in the United States with 5 years of experience who uses AI tools. Their current stack is Python, JavaScript, SQL, PostgreSQL and Docker.

The peer group:

- **Pool size.** All four filters hold, so nothing is relaxed: 731 developers. The 300 most similar are the neighbours.
- **Peer pay.** The group's median pay is $130,000. Its premium is −2.0%, meaning these peers earn slightly *less* than the average developer with the same experience in the same country.
- **Persona.** Lean generalist stacks.

Recommendations:

#	Skill	Peers who know it	Peers who want it	Stand-alone pay association	p
1	TypeScript	67%	47%	+6.7%	< 0.001
2	VS Code	68%	44%	+3.2%	0.003
3	npm	63%	27%	+4.2%	< 0.001
4	React	56%	32%	+5.1%	< 0.001
5	AWS	51%	23%	+21.4%	< 0.001

For TypeScript the score is about $0.671 + 0.5 \cdot 0.065 + 0.3 \cdot 0.472 + 0.05 \approx 0.895$. The picks are the skills this developer's closest peers overwhelmingly use and want, with AWS carried up by its large pay association.

Plan valuation. The joint regression (§5.6) over these five skills gives a combined association of **+23.7%** (95% CI +19.8% to +27.7%, $p \approx 4 \times 10^{-39}$). Their stand-alone effects sum to about +41%, so jointly much of the overlap disappears.

The breakdown shows why. AWS keeps its +21.4%, while TypeScript drops to +4.3%. VS Code and React fall to about zero once the rest of the bundle is held fixed ($p = 0.50$ and 0.88). npm turns slightly negative (−3.2%). These skills are mostly *relevance* picks: they fit the stack and are in demand, but they are not separate sources of pay.

This is the kind of nuance the app surfaces: the ranking says what fits, and the calculator says what it is worth.

8. The application

`app/streamlit_app.py` is an interactive Streamlit app ("Star Atlas" design: dark navy chart ground, chalk ink, amber reserved for the five recommendations).

Area	Content	Model behind it
Left rail	Role, years of experience (0–50), country (all), AI-tool use, current stack in six skill groups	inputs
Header line	Size and description of the comparison group, and any relaxed filter	§5.13, step 1
Sky chart	All 122 skills on the t-SNE map. Your stack is drawn as a constellation, the five picks are ringed in amber, and star size reflects salary effect	§5.14
Plan panel	The joint uplift of the five picks with its 95% CI, the ranked list, and movement tags (up / down / new) when an input changes. Facts: group median pay, premium versus the same country and experience, persona	§5.6, §5.13
Index of evidence	Each pick's peer share, demand share, pay association, CI, p and satisfaction coefficient	§5.5, §5.8, §5.13
Persona	The assigned persona and its distinguishing skills	§5.9
ROI landscape	Every skill's pay association as a point on its 95% interval, brightness by significance	§5.5
What-if calculator	Pick any skills and see their joint uplift and per-skill joint coefficients	§5.6
Method notes	Plain-language descriptions of the models and their caveats	

Every input changes the output. Role, country, experience and AI use change the peer pool, and so the shares, the median pay, the premium and the ranking. The stack changes the neighbours, the persona and the candidate set.

Live application: https://skillet.streamlit.app/

Run it locally:

```
.venv/bin/python -m streamlit run app/streamlit_app.py
```

9. Model validity and the corrections made

An audit of an earlier version of the project found and fixed several modelling errors. They are recorded here because they change the conclusions.

Issue	Earlier version	Fix	Effect
Currency	Modelled `CompTotal` (local currency) with dummies for only the 10 largest countries	USD pay and a fixed effect for every country	SLR R^2 0.002 → 0.128. 12 skill coefficients changed sign; significant skills rose from 75 to 98
Plan uplift	Summed stand-alone coefficients and summed their interval bounds	One joint regression and a contrast test (§5.6)	e.g. +152% → +57%. The interval is now valid
Constant score term	Score included a peer salary premium	Removed (the same for every candidate, so it cannot change the ranking); now shown as context	Clearer model
Inert inputs	Some profile inputs did not affect peers	Profile filter with reported relaxation (§5.13)	Every input moves the result
Unweighted peers	Small pools ignored the stack	Similarity weighting (§5.13, step 3)	Stack matters at every pool size
Persona	App used a heuristic label	Saved scaler → PCA → K-means model	App persona matches the analysis
Trend bug	Salaries paired with other respondents' skills after an index reset	Index-aligned fit	Corrected the "risers" story (Docker fell, rather than Jupyter and Flask rising)
Categorical dummies	Remote-work and org-size dummies matched 2023 answer text, so org dummies were all zero	Exact 2025 answers	Valid satisfaction features
AI pay gap	Raw t-test only	Added the controlled OLS	Revealed the Simpson's-paradox reversal
Recommender	Never evaluated	Held-out evaluation with baselines (§5.15)	Evidence that it works

10. Recommendations for decision makers

Individual developers.

- Use the app with your real profile. Read the *joint* uplift, not the per-skill numbers added up.
- Prefer picks whose interval excludes zero.
- Skills with high peer adoption but no joint pay effect (e.g. VS Code) are about fitting in with your peers, not about pay.

L&D managers.

- Treat each persona as a training track and fund the high-ROI, high-relevance skills within it.
- Cloud and data-platform skills (Terraform, AWS, Snowflake, Databricks, Kubernetes) carry the largest and most stable premiums across four years.
- Skills that are becoming mainstream (Docker, Redis) are worth teaching for relevance, but their pay premium is shrinking.

HR and analytics teams.

- Always control for country (and experience) before comparing pay groups. The AI-use example shows the sign can flip otherwise.
- Re-run the pipeline every year to track drift.

11. Assumptions and limitations

1. **Association, not causation.** All regression coefficients are conditional associations. A developer who learns Scala will not automatically earn 35% more. Scala users may differ in unobserved ways: employer type, seniority within a level, or industry. Country and experience are controlled; selection is not.
2. **OLS assumptions.** - The model assumes linearity in log pay and independent errors. - Standard errors are classical. Heteroskedasticity-robust (HC) standard errors would be a sensible extension; with n ≈ 18,000 the main conclusions are unlikely to change. - The residuals of log pay are approximately but not exactly normal. Inference relies on the large-sample central limit theorem.
3. **Survey sample.** Stack Overflow respondents are self-selected and skew towards some countries and roles. Pay is self-reported.
4. **Currency.** USD at the survey's exchange rates, not purchasing-power adjusted. Country fixed effects remove level differences, but not differences in how strongly skills are rewarded in each country.
5. **Cross-section.** A single year per model. The trend compares different respondents each year.
6. **Recommender design choices.** - The weights (0.5, 0.3, 0.05, 0.02), k = 300, the 50-peer minimum and the ±10-year window are judgement calls. - The weights were checked against held-out data (§5.15); the others were not tuned.
7. **Satisfaction.** Weakly predictable (AUC ≈ 0.57). The satisfaction coefficients are shown but not used in the score.
8. **Clustering.** Low silhouette (0.085). The personas are a useful summary, not sharply separated groups.

Future work:

- purchasing-power-adjusted pay;
- robust or clustered standard errors;
- per-country skill slopes (interaction terms);
- panel or causal designs on respondents with work histories;
- learning the score weights directly (learning-to-rank).

12. Reproducibility

```
make data        # download the 2025 survey (skips files already present)
make all         # full pipeline: preprocess, tests, regressions, classifiers, clustering,
evaluation, FA, ANOVA, network, sky map
make trend       # optional: the four-year trend (downloads 2022-2024)
make app         # launch the Streamlit app
```

`scripts/smoke_recs.py` checks the recommender end-to-end. Fixed seeds (42) are used throughout, for the splits, K-means, t-SNE and bootstraps.

13. Appendix

13.1 Syllabus mapping

Module	Topic	Where
M1	Data cleaning, feature engineering	§3, `src/preprocess.py`
M2	Visualisation; hypothesis tests (t, χ^2, Cramér's V)	§4, §5.7, `src/stats_tests.py`
M3	Simple and multiple regression, fixed effects, VIF, contrasts	§5.3–5.6, `src/regression.py`, `src/salary_calc.py`
M4	Logistic regression, decision tree, Naive Bayes, ROC/AUC, KNN	§5.8, §5.13, `src/classification.py`, `src/recommend.py`
M5	PCA, K-means, hierarchical clustering, factor analysis	§5.9–5.10, `src/cluster.py`, `src/fa.py`
M6	Network analysis	§5.12, `src/network.py`
M7	ANOVA, model evaluation, decision support	§5.11, §5.15, `src/anova.py`, `src/evaluate.py`, app

13.2 File map

File	Role
`src/preprocess.py`	Survey → `data/processed.parquet`
`src/want.py`	Want-to-work-with extract → `data/want_sets.parquet`
`src/stats_tests.py`	Hypothesis tests → `data/stats_tests.json`
`src/regression.py`	SLR, MLR, per-skill ROI → `data/skill_roi.csv`, `data/regression_summary.json`
`src/salary_calc.py`	Joint plan uplift (§5.6)
`src/classification.py`	Satisfaction models → `data/satisfaction_*`
`src/cluster.py`	PCA + K-means → `data/cluster_model.joblib`, `data/cluster_meta.json`
`src/fa.py`, `src/anova.py`, `src/network.py`	Factor analysis, ANOVA, network
`src/recommend.py`	The recommender (§5.13)
`src/skymap.py`	t-SNE skill map → `data/skill_map.json`
`src/evaluate.py`	Held-out evaluation → `data/recommender_eval.json`
`src/trend.py`	2022–2025 trend → `data/trend_roi.csv`, `data/trend_summary.json`
`app/streamlit_app.py`	The interactive app